ARCADE HUB

Technical Documentation | v1.0.1

Framework	React Native 0.81.5 + Expo SDK 54
Platform	Android (APK/AAB) + Web Preview
Games	7 Classic Mini-Games
Test Coverage	97 Passing Tests (Unit + Property-Based)
License	MIT (Open Source)
Latest Release	v1.0.1 (March 18, 2026)

github.com/aryannten/Arcade-Hub

1. Project Overview

Arcade Hub is a fully offline, cross-platform mobile gaming application built with React Native and Expo. Marketed as "your pocket-sized gaming paradise", it bundles seven classic mini-games inside a single installable Android APK, unified by a modern neon glassmorphism UI and a shared progression system covering stats tracking, achievements, and dark/light theming.

The project is entirely open-source under the MIT license and demonstrates a production-quality React Native architecture complete with a custom design-token system, async persistent storage, haptic feedback, property-based testing, and an EAS (Expo Application Services) build pipeline.

Repository: https://github.com/aryannten/Arcade-Hub

APK Download: https://github.com/aryannten/Arcade-Hub/releases/tag/v1.0.1

2. Goals & Scope

2.1 Primary Goals

- Bundle classic mini-games in one compact, offline-capable Android app.
- Deliver a premium, consistent visual identity through a centralised design system (tokens + reusable components).
- Track individual game statistics and award achievements to increase player engagement.
- Demonstrate production-level React Native patterns suitable for a portfolio or learning reference.
- Validate core game logic with a strong automated test suite (97 tests, including property-based tests).

2.2 Out of Scope (v1.0.1)

- Multiplayer / real-time networking features.
- iOS build (configuration present in app.json, but not distributed).
- Backend server, user accounts, or cloud-synced progress.
- In-app purchases or ads.

3. Technology Stack

Category	Technology	Version / Notes
Core Framework	React Native	0.81.5
Runtime	Expo SDK	54 (newArchEnabled: true)
UI Layer	React	19.1.0
Gradients	expo-linear-gradient	^55.0.9
Persistent Storage	@react-native-async-	2.2.0

	storage/async-storage	
Haptics	expo-haptics	~15.0.8
Audio	expo-av	~16.0.8
Safe Area	react-native-safe-area-context	~5.6.0
Web Target	react-native-web	^0.21.0
Testing (runner)	Jest	^30.3.0
Testing (PBT library)	fast-check	^4.6.0
Testing (component)	@testing-library/react-native	^13.3.3
Build / Distribution	EAS (Expo Application Services)	CLI >= 3.0.0
Transpiler	Babel (babel-preset-expo)	^55.0.11
Language	JavaScript (JSX)	100% (no TypeScript)

The new React Native Architecture (Fabric renderer + JSI) is enabled via the

The project enables the `newArchEnabled: true` flag in app.json, meaning it targets React Native's new Fabric renderer and JSI bridge for improved performance.

4. Project Structure

The repository follows a feature-first directory layout. All source code lives under src/, while configuration and entry points reside at the root.

```
Arcade-Hub/
├── App.js                      # Root component – hub screen & navigation
├── index.js                    # Expo entry point (registers App component)
├── app.json                    # Expo / EAS configuration
├── eas.json                    # EAS build profiles (dev / preview / prod)
├── babel.config.js             # Babel preset (babel-preset-expo)
├── jest.config.js              # Jest configuration
├── package.json                # Dependencies & npm scripts
│
├── assets/                     # Static assets
│   ├── app-icon2.png           # App icon (adaptive + splash)
│   └── games/
│       ├── flappy-bird.png     # Bird sprite (Flappy Bird game)
│       └── snake-food.png      # Food sprite (Snake game)
│
├── src/
│   ├── components/             # Shared UI components
│   │   ├── GameCard.jsx        # Hub grid card per game
│   │   └── InstructionsModal.jsx   # Per-game instruction overlay
│   │
│   ├── design/                 # Design system
│   │   ├── tokens.js           # Design tokens (colors, spacing, type, gradients)
│   │   └── components/         # Reusable design primitives
│   │       ├── GlassCard.jsx
│   │       ├── GradientButton.jsx
```

```
│   │       ├── StatBar.jsx
│   │       ├── DifficultyBadge.jsx
│   │       └── NeonText.jsx
│   │
│   ├── games/                  # Individual game screens
│   │   ├── MemoryGame.jsx
│   │   ├── ReactionTest.jsx
│   │   ├── NumberGuesser.jsx
│   │   ├── RockPaperScissors.jsx
│   │   ├── TicTacToe.jsx
│   │   ├── Snake.jsx
│   │   └── FlappyBird.jsx
│   │
│   └── utils/                  # Utility modules
│       ├── storage.js          # AsyncStorage wrapper (load/save stats)
│       ├── theme.js            # Theme context + dark/light management
│       ├── sounds.js           # Haptic feedback helper
│       └── achievements.js     # Achievement definitions & unlock logic
│
├── __mocks__/                  # Jest manual mocks (Expo/RN modules)
│
└── __tests__/                  # Test suites
    ├── unit/                   # Unit tests per util/component
    └── property/               # Property-based tests (fast-check)
```

5. Key Files Reference

5.1 App.js – Hub & Navigation

App.js is the root component of the application. It acts as both the game hub screen (the grid of game cards a player sees on launch) and the top-level navigation controller. Because the project has no dedicated navigation library (e.g., React Navigation), routing is handled with React state: a currentGame state variable holds either null (show hub) or a game identifier string (show that game's screen). The component is responsible for:

- Rendering the neon-styled hub header with app title and theme-toggle button.
- Displaying the search bar and difficulty filter controls.
- Mapping the games array to GameCard components in a two-column grid.
- Conditionally rendering the selected game screen component.
- Injecting the global ThemeContext down the component tree.

5.2 index.js – Expo Entry Point

A minimal file that imports App and registers it with Expo's AppRegistry. No application logic lives here.

5.3 app.json – Expo Configuration

Defines the Expo app manifest consumed at build time:

Key	Value / Purpose
name	Arcade Hub
slug	mini-games-app (used internally by EAS)
version	1.0.0

orientation	portrait (locked)
icon	./assets/app-icon2.png
userInterfaceStyle	automatic (respects system dark/light mode)
newArchEnabled	true (Fabric + JSI)
splash.backgroundColor	#0a1628 (deep navy to match UI)
android.package	com.arcadehub.minigames
android.edgeToEdgeEnabled	true (draws behind nav bar)
extra.eas.projectId	0cc9a9da-100c-4e21-9094-c77df1d57b79

5.4 eas.json – Build Profiles

Configures the three EAS build profiles used to produce installable binaries:

- development – internal distribution with developmentClient enabled; for iterative testing on real devices with Expo Dev Client.
- preview – internal distribution, outputs a plain .apk file ideal for sharing with testers without Play Store involvement.
- production – outputs an .aab (Android App Bundle) ready for Play Store submission.

6. Design System

One of Arcade Hub's most notable engineering decisions is its comprehensive design system, housed under src/design/. Rather than scattering style values inline across game files, the project extracts every primitive into shared tokens and reusable components – a pattern more commonly seen in web design systems like Tailwind or Material UI.

6.1 Design Tokens (src/design/tokens.js)

tokens.js is the single source of truth for all visual values:

Token Group	Contents
Colors.background	Deep navy (#0A1628) — app & screen background
Colors.surface	Slightly lighter navy (#1A2844) — card surfaces
Colors.neon.cyan	#00D4FF — primary neon accent
Colors.neon.purple	#A855F7 — secondary neon accent
Colors.neon.amber	#F59E0B — warning / RPS game
Colors.neon.red	#EF4444 — danger / Tic Tac Toe game
Gradients.*	Per-game 2-stop linear gradients (7 total)
Spacing.*	xs / sm / md / lg / xl / xxl scale
Typography.*	Custom font families and size scale
Shadows.*	Neon glow shadow configs for depth / emphasis

6.2 Reusable Design Components

Five primitive components in src/design/components/ form the building blocks for every game screen:

Component	Purpose & Behaviour
GlassCard.jsx	A View with semi-transparent background, blur effect, and rounded corners to produce the glassmorphism aesthetic. Wraps other content.
GradientButton.jsx	A pressable button backed by expo-linear-gradient. Accepts a gradient key (mapped from tokens), a label, and an onPress handler. Provides haptic feedback on press.
StatBar.jsx	A labelled horizontal display row — icon / label on the left, value on the right — used in game result screens and the global stats panel.
DifficultyBadge.jsx	A small pill-shaped badge colour-coded by difficulty level (Easy / Medium / Hard). Used on GameCard and inside game headers.
NeonText.jsx	A Text component wrapped with a neon glow shadow effect derived from tokens.Shadows. Used for headings and score displays.

6.3 Theme System (src/utils/theme.js)

theme.js implements a React Context that provides the current theme object (dark or light) and a toggle function to every component in the tree. The selected theme is persisted to AsyncStorage so it survives app restarts. Components read theme values via a custom useTheme() hook rather than importing raw token values, which allows future theme additions (e.g., OLED black) without touching game files.

7. Games

Each game is a self-contained React Native screen component located in src/games/. All games share the same lifecycle pattern: on mount they load the player's stored best stats via the storage util; on game-over they write updated stats and check for newly unlocked achievements.

Game	Difficulty	Gradient Theme	Core Mechanic
Memory Match	Medium	Purple neon	Flip cards on a 4x4 grid; match all pairs in fewest moves and shortest time.
Reaction Test	Easy	Green gradient	Tap when the screen turns green. Measures tap latency in milliseconds. Tracks best reaction time.
Number Guesser	Easy	Blue gradient	Binary-search style: guess a hidden number 1–100. Told Higher / Lower after each guess. Fewest attempts wins.
Rock Paper Scissors	Easy	Amber gradient	Best-of-N hand game against a computer random opponent. Tracks win rate.
Tic Tac Toe	Medium	Red gradient	Classic 3x3 grid; play vs computer (AI opponent) or vs a second player locally.
Snake	Hard	Cyan gradient	Directional swipe controls. Eat food sprites (custom PNG), grow longer, avoid walls and self. Score = length.

Flappy Bird	Hard	Light blue gradient	Tap to flap; navigate through procedurally-spaced pipes. Uses a custom bird PNG sprite. Score = pipes cleared.

7.1 Image-Asset Games

Snake and Flappy Bird are the only two games that use external image assets beyond the app icon. The sprites reside at assets/games/ and are loaded via React Native's Image component with require(). Keeping sprites as PNG rather than SVG was a deliberate choice to avoid an additional vector-rendering dependency.

7.2 Shared Components Used by Games

- GameCard.jsx – rendered by App.js for each game in the hub grid. Displays game name, difficulty badge, gradient icon area, and a short tagline. Tapping navigates to the game screen.
- InstructionsModal.jsx – an overlay triggered by a help icon on each game screen. Receives a game-key prop and renders the relevant instruction copy. Dismissible with a tap outside or a close button.

8. Utility Modules

8.1 storage.js

A thin abstraction over @react-native-async-storage/async-storage. Provides two primary helpers:

- loadStats(gameKey) – reads and deserialises a JSON blob from AsyncStorage for the given game key, returning a typed stats object with sensible defaults if no data exists.
- saveStats(gameKey, stats) – serialises and writes the stats object back to storage. Called at game-over to persist the session result.

By centralising all storage reads/writes here, game components remain ignorant of AsyncStorage's API, making it trivial to swap the persistence layer (e.g., SQLite) in the future.

8.2 achievements.js

Defines the full achievement catalogue as a static array of achievement definition objects. Each object specifies:

- id – unique string identifier.
- title / description – display strings.
- icon – emoji icon for the achievement notification.
- check(stats) – a pure function that receives the current game stats object and returns true if the achievement should be unlocked.

The module also exports an unlockAchievements(gameKey, stats) function that evaluates all checks, compares against the already-unlocked set in storage, persists any newly earned achievements, and returns the list of new unlocks for the game screen to display as a notification.

8.3 sounds.js

Despite its name, sounds.js primarily wraps expo-haptics rather than audio playback (expo-av is included as a dependency for potential future audio). It exports named helper functions — tapFeedback(), successFeedback(), errorFeedback() — each triggering a different haptic impact style, allowing game logic to call semantic methods without knowing the underlying Haptics API details.

9. Stats & Achievement System

9.1 Stats Tracked Per Game

The stats schema varies per game but generally includes:

Stat Field	Games That Track It
gamesPlayed	All games
gamesWon	Memory, RPS, Tic Tac Toe, Number Guesser
bestScore	Snake, Flappy Bird
bestReactionTime	Reaction Test
minAttempts	Number Guesser
bestMoves / bestTime	Memory Match

9.2 Achievement Examples

- First Win – win any single game.
- Speed Demon – complete Reaction Test in under 200 ms.
- Perfect Memory – complete Memory Match without a single mismatch.
- Marathon Player – play a total of 50+ games across all titles.
- Snake Master – reach a snake length of 20+.
- Pipe Pro – clear 10+ pipes in Flappy Bird.

10. Testing

Arcade Hub ships with 97 passing tests split across two strategies, which is unusually thorough for a personal mobile project.

10.1 Unit Tests (__tests__/unit/)

Standard Jest + @testing-library/react-native tests covering:

- Utility function correctness (storage serialisation/deserialisation, achievement unlock logic, theme context transitions).
- Component snapshot tests for GameCard and InstructionsModal.
- Edge-case game logic (e.g., Snake collision detection, Tic Tac Toe win detection).

10.2 Property-Based Tests (__tests__/property/)

Property-based testing (PBT) with fast-check generates hundreds of arbitrary inputs and asserts that game invariants hold for all of them. This is significantly more powerful than hand-picked examples for numeric logic. Examples of properties tested:

- Number Guesser: for any secret number in [1, 100], the higher/lower feedback function always converges correctly within at most 7 guesses in an optimal strategy.
- Storage: serialise then deserialise any stats object always yields the original object (roundtrip identity).
- Reaction Test: timestamp deltas are always non-negative and within plausible human reaction bounds.

10.3 Manual Mocks (__mocks__/)

The __mocks__/ directory contains Jest manual mocks for Expo and React Native modules (expo-haptics, expo-av, @react-native-async-storage/async-storage, etc.) that would otherwise fail in a Node test environment since they rely on native device APIs. All mocks are no-ops or return Promise.resolve() stubs.

To run the test suite:

```
npm test
# or
npx jest --watchAll
```

11. Build & Distribution

11.1 Local Development

```
git clone https://github.com/aryannten/Arcade-Hub.git
cd Arcade-Hub
npm install

# Start Metro bundler
npm start          # or: npx expo start

# Open on Android emulator
npm run android

# Open on iOS simulator (macOS only)
npm run ios

# Open in browser (Expo Web)
npm run web
```

11.2 EAS Cloud Builds

```
# Install EAS CLI
npm install -g eas-cli

# Preview APK build (Android, internal distribution)
eas build --profile preview --platform android

# Production App Bundle (Play Store)
eas build --profile production --platform android
```

The EAS projectId in app.json (0cc9a9da-100c-4e21-9094-c77df1d57b79) ties the build to the Expo account that owns this project. Collaborators will need to either link their own EAS account or obtain project access.

11.3 Installing the Pre-Built APK

- Download Arcade-Hub-v1.0.1.apk from the Releases page on GitHub.
- On the Android device, go to Settings > Security > Install from Unknown Sources and enable it.
- Open the downloaded APK file and follow the install prompts.
- No Play Store account or internet connection is required after installation.

12. Configuration Deep-Dive

12.1 package.json – Dependency Notes

The project declares no peer-dependency conflicts with Expo SDK 54 as of v1.0.1. Notable points:

- react-dom and react-native-web are listed in dependencies (not devDependencies) – required because Expo Web is a supported target.
- fast-check is a devDependency used exclusively in the test suite; it adds zero size to the production bundle.
- There is no react-navigation dependency. The custom App.js state-based router is the sole navigation mechanism, keeping the bundle lean.

12.2 Android-Specific Settings

- Package name com.arcadehub.minigames – follows standard reverse-domain notation.
- Adaptive icon – uses app-icon2.png with a #0A1628 background colour, matching the splash screen for a seamless cold-start experience.
- Edge-to-edge true – the app draws content behind the Android navigation bar for an immersive full-screen feel.
- Tablet support – supportsTablet: true is set on the iOS config; Android inherits tablet support by default.

13. Contributing

Arcade Hub is open source under the MIT License. Contributions of all sizes are welcome.

13.1 Adding a New Game

- Create a new file src/games/YourGame.jsx as a self-contained React Native screen component.
- Define a game metadata object (name, difficulty, gradient key, tagline) and add it to the games array in App.js.
- Add an entry to the instructions map used by InstructionsModal.jsx.
- Add storage keys and stat defaults in storage.js.
- Define achievement check functions in achievements.js.
- Write at least one unit test and one property-based test before submitting a PR.

13.2 Reporting Bugs

Open an issue on GitHub. Include device model, Android version, reproduction steps, and any console output or screenshots.

14. Release History

Version	Date	Notes
v1.0.1	March 18, 2026	Latest stable release. APK available on GitHub Releases. 14 commits, all games present and 97 tests passing.
v1.0.0	~March 2026	Initial public release.

15. Quick Reference

Command / Resource	Description
npm start	Start Expo Metro bundler
npm run android	Launch on Android emulator / connected device
npm test	Run all 97 Jest tests
eas build --profile preview	Build preview APK via EAS cloud
eas build --profile production	Build production AAB via EAS cloud
GitHub Repo	github.com/aryannten/Arcade-Hub
Releases Page	github.com/aryannten/Arcade-Hub/releases
APK (v1.0.1)	github.com/aryannten/Arcade-Hub/releases/tag/v1.0.1

— End of Documentation —